Branch `rwc1`: changes to the end-to-end attack

full_end_to_end/ · 26 commits since `main` plus one uncommitted change · 14 October 2026

1. Summary

On `main`, the pipeline only worked on the original target, `unitary_32_32x3_10_float64.keras`: 32 inputs, three hidden layers of 32 neurons and 10 classes. This branch makes it run on any Dense ReLU network. It has been tested against two new targets:

- `data/test.keras`: 32 inputs, five hidden layers of 16 neurons, 4 classes.
- `data/ids_111_64x4_4_float64.keras`: 111 inputs, four hidden layers of 64 neurons, 4 classes.

Generalizing the code exposed problems that never occur on the original target. That network is wide and shallow, and most of its neurons are active most of the time. The new targets are deeper, and many of their neurons are inactive at most points. When a neuron in an earlier layer is inactive (always off) at every dual point, its weight into the next layer can't be determined from those points. The pairwise clustering test also starts accepting dual points from the wrong neurons. Several kinds of false clusters appear: deeper neurons, faces of the decision boundary, and linear identities among the earlier layers' outputs.

The sign-recovery code from the paper failed for a different reason. It assumed that the local matrix at every dual point has full rank, and on these networks that is often false.

The changes fall into seven groups:

1. Generalization to arbitrary architectures.
2. Many fixes to clustering correctness.
3. Recovering weights that clustering can't see.
4. A new sign-recovery method.
5. Cutting oracle queries.
6. Checkpointing and progress reporting for long runs.
7. Validation that tolerates weights that are still unknown.

Terminology. A *dual point* is a point on the decision boundary where the boundary bends, because some neuron changes state there. A *walk* is `WALK_PATHS` paths that follow the decision boundary and record each bend as a dual point (`find_duals.py`). *Clustering* groups dual points by neuron: a *seed* dual point recruits other dual points that pass a pairwise test, and the group is then fit to one hyperplane behind the layers already recovered (the *prefix*). *Refine* re-fits each row to about 1e-14 from fresh kinks, and *signs* decides which side of each row is the active side.

2. Generalizing beyond the original target

2.1 Hard-coded architecture

Problem. The code hard-coded the input size, the width of every layer and the class count (`IDIM = DIM = 32`, `HIDDEN_LAYERS = 3`, `NCLASS = 10`, `reshape(-1, 32)`, and `[IDIM, DIM, DIM][layer]`). Any other model crashed or was silently misread.

Fix.

- `oracle.py` reads the model named by `ORACLE_MODEL`, which worker processes inherit, and parses the activations from the model's config. `oracle.architecture()` exposes only the public shape (no weights).
- `run.py` takes `--model` and `--out`, writes to `out/<model>/`, and refuses to mix two models' outputs in one directory (`model.json`).
- Widths, input dimension and class count come from `architecture()` everywhere: `recover_weights` (`Prefix.out_dim`), `recover_head`, `validate` and `net.npz` with any number of layers.

f1ecbdb

2.2 Wrong input dimension and too few kinks in refine

Problem. Refine took the input dimension from the width of the row it was refining. That is only right for layer 0, so deeper layers crashed with a reshape error. On top of that, 64 kinks can't pin down a row with 111 weights.

Fix. `refine.input_dim()` takes the dimension from the first prefix layer. The kink count is now `max(KINKS, 2 × row length)`.

70e4d8b, 975ad73

2.3 Head with more unknowns

Problem. A wider top layer means more unknowns in the head, so the fixed number of boundary segments was sometimes not enough to fit it.

Fix. `head()` doubles the number of segments, up to 3 times, until the head passes its self-check.

f1ecbdb

3. Clustering: issues that blocked or corrupted layers

Most of the work went into `par_cluster.py`, and each fix below answers a specific failure seen on `test.keras` or the ids model.

In outline, a cluster is accepted only if it passes all of these checks:

- a hyperplane fits its dual points (relative residual and gap);
- the bias is consistent across its members;
- it is not an identity, not disengaged and not a face;
- it is not a duplicate of a cluster already found;
- it is not a deeper neuron in disguise.

Issue	Why it happened	Change
Fit threshold fixed in absolute terms 3903433	The test `S[-1] > 1e-6` depends on how the duals are scaled, so it was too strict for some layers and too loose for others.	The threshold is now relative: `S[-1] < 1e-7·S[0]`.

Deeper neurons accepted as this layer's 294b718, c32452a	Within one region where this layer's neurons don't change state, the layer is a single linear map. A deeper neuron's zero set is then a flat hyperplane behind the prefix, and it solves exactly like a real neuron.	`disguised()` runs once the layer looks complete. It rejects a cluster whose duals are split by fewer than `SPLIT_MIN` = 25% of the other clusters' hyperplanes. Rejected clusters are kept, so the same hyperplane is not accepted again.
Neurons that are off at every dual eb7ae5a, 381687b	When an earlier neuron is off at every dual of a cluster, its weight has no effect there. The old code rejected such clusters outright when `layer > 0`, so many neurons of deep layers could never be clustered.	The fit leaves those columns out and returns their weights as exactly 0, meaning unknown; refine fills them in later (§4). With fewer unknowns, wrong groups fit more easily, so a cluster with unknown weights needs `MIN_CLUSTER_ZEROED` = 8 members. `is_duplicate` compares two clusters only on the weights both of them know.
Linear identities among the prefix's outputs eb7ae5a, 381687b	Where the active outputs of the earlier layers depend on fewer neurons further down than there are of them, those outputs satisfy an exact linear relation. Duals of any neuron in that region fit the relation, with a common bias.	`prefix_identity()` moves each dual off its point within the same on/off pattern. If the hyperplane still holds at at least `IDENTITY_SHARE` = 25% of the duals, the cluster is rejected.
Rows that weigh only inactive inputs a374b2a	The equation `w·h + b = 0` holds trivially wherever every input that the row weighs is off. Such a row fits the duals of any neuron there; ids layer 2, cluster 61 was one.	`disengaged()` rejects a row that puts less than `ENGAGED_MIN` = 1e-3 of its weight on the inputs that are active at one of its duals.
Faces of the decision boundary e17768f	A flat piece of the decision boundary contains the bend lines of every bend on its edges, so their duals fit it exactly, like a cluster.	`is_face()` flags a hyperplane whose input-space normal equals one of the boundary normals measured at most of its duals (`FACE_TOL` 1e-6, `FACE_SHARE` 0.5). That face's duals are excluded and the seed keeps growing, up to 3 faces.
One stray member ended the seed a643363, 381687b, c512c78	In deep layers, two duals often have few active coordinates to disagree on, so the pairwise test lets in duals of other neurons. A single stray makes the whole set fail, and the seed was abandoned. This stopped ids layer 2, neuron 16 from clustering.	`prune()` fits random subsets of sizes 3, 4 and 6 that include the seed. It keeps the members that lie on the best of those hyperplanes (`PRUNE_TOL` 1e-5) and keeps growing, up to 3 times per seed. A cluster of three or fewer waits for one more member before pruning, because any smaller subset fits trivially.
Real members discarded as repeats 381687b	Duals whose boundary normals had a dot product above 0.8 were treated as the same bend found twice. A neuron that barely affects the decision bends the boundary only slightly, so its genuine duals were dropped. This blocked test.keras layer 3, neuron 6.	Two duals now count as the same bend only if `|cos| > SAME_NORMAL` = 1 − 1e-6.
A rejected impostor blocked a real neuron 381687b	Within one region, a deeper neuron's row is a combination of this layer's rows. It can sit within `|cos|` 0.8 of a real neuron, so rejecting it also blocked the real neuron.	A new cluster is blocked by a rejected one only if they are the same hyperplane: `SAME_ROW` = 1 − 1e-6.

Hyperplanes the duals barely determine 84c55a5	With many unknown weights and few duals, the null space has just one direction, and duals of several neurons can fit it by chance. ids layer 2, cluster 49 was one.	The fit must also show a clear gap, `S[-2]/S[0] ≥ GAP_MIN` = 0.007. If it doesn't, the cluster keeps growing instead.
Rare neurons never clustered 381687b → 31d764d	A neuron that is almost always on or almost always off has few duals per walk. Seeds were also only paired with duals near their own walk, and 1,000 partners at most.	First, a second pass over the leftover duals. This was later replaced by staged walks (§6): clustering starts with 5 walks and adds one at a time while neurons are missing, up to `MAX_WALKS_CLUSTERING` = 16. Each seed's partners come from all walks loaded so far, alternating between them, and the 1,000-partner limit is gone.

4. Recovering weights the clustering cannot see

4.1 Unknown weights (inputs that are off at every dual)

Problem. A row that clustering left with zeroed, i.e. unknown, weights is only correct where those inputs are off. Refine then followed the wrong hyperplane, or fitting failed.

Fix.

- `recover_weights` solves only the columns that have data (the free columns).
- `refine.complete_row()` starts from boundary points on the row's hyperplane where some of the missing inputs are on. From each one, `far_kink()` marches outward to the nearest real bend, which gives one linear equation in the missing weights.
- Kinks that lie on a hyperplane already known are dropped. The kinks of deeper neurons can't be recognized one at a time, so `consensus()` keeps the solution that the most equations agree with exactly (`AGREE` = 2 more equations than unknowns).

eb7ae5a

4.2 Weights the refit cannot constrain

Problem. A prefix unit that is on at no kink, or at very few, makes the fit's matrix rank-deficient.

Fix. `fit_row` leaves out any unit seen at fewer than `SEEN_MIN` = 3 kinks. If the null space is still unclear, it also drops the least-supported unit, and the dropped units keep their old weights. Weights that came from `complete_row` are then solved again from its anchor kinks, now against the refined row.

c32452a, eb7ae5a

4.3 Wrong completions (ids layer 1, rows 51 and 58)

Problem. `consensus` scaled its tolerance by `|a|`, so a large, wrong solution loosened its own test. The kinks then followed the wrong hyperplane.

Fix.

- The tolerance no longer depends on `a`.
- If the refit moves the weights clustering already knew by more than `KNOWN_MOVED` = 1e-6 (as 1 − |cos|), the completion is treated as wrong and retried, up to `COMPLETE_ATTEMPTS` = 3.
- A fit far from round-off is rejected.

4cfc2e8

5. Sign recovery

5.1 Hangs

Problem. `findDualPoints` retried forever when no dual point could be found, and a tied vote raised an assertion.

Fix. It now gives up after 1,000 failures in a row. A vote that stops early still decides by majority but is reported as not confident; a tie raises an informative error. Progress is printed for each neuron.

66cea22

5.2 The original method assumed full-rank dual points

Problem. The paper's method walks to the next bend on each side and compares the distances. That relies on the local matrix `M` having full rank at the dual point. In deep, mostly-inactive networks it often doesn't, and on `test.keras` two signs came out wrong even though every row was right.

Fix. One experiment, at each dual point of neuron *j*:

- The probe is `v = e_j·pinv(M)`: an input step that moves neuron *j*.
- The decision boundary's normal is measured on both sides of the neuron's hyperplane.
- The side whose normal has the larger `|cos|` with `v` is voted the active side, because where the neuron is off, the boundary ignores `v`.

A majority vote over many dual points absorbs the experiments where `M` is singular.

4b6d025

5.3 Errors in the vote from other neurons

Problem. Where `M` is singular, `v` also moves other neurons of the same layer, which biases the vote.

Fix. Every experiment keeps `M`, `z` and both measured normals. Later passes use the current sign estimates to work out which of the layer's neurons are active at each dual point, and recompute the probe as `e_j·pinv(M_active)`. Each pass then re-votes using those stored measurements, without new queries. Passes repeat until no sign changes, up to `SIGN_PASSES_MAX` = 6, because fixing one neuron can fix the next.

c32452a, 3ab6416

6. Oracle queries, speed and long runs

Issue	Change
Sign queries 3ab6416	The walk already measured the boundary normals on both sides of every bend, so each dual point that lies on a row's hyperplane (within `STORED_DIST` = 1e-6 in input space) is a free sign experiment. These stored experiments are used first. New dual points are bought only if fewer than `STORED_ENOUGH` = 50 stored ones exist and the vote isn't confident yet. With the uncapped walks, layers 0–2 needed no sign queries at all.
Slow clustering 84c55a5	• Each worker loads every walk once and caches the prefix outputs (`walk_pool`). • Seeds and partners that lie on a hyperplane already found or rejected are skipped with one matrix product, with no pairwise test. • Partners alternate between walks (`partners`).

Walks were most of the cost 0a12ae6 → 31d764d, 638405a	Making enough walks up front for the rarest neuron cost 32 walks, which was 73–97% of all queries. Now only `WALKS` = 5 are made up front. A layer's clustering adds one walk at a time, each tried as seeds against every walk so far, until the layer is complete or `MAX_WALKS_CLUSTERING` = 16 is reached. A walk added for one layer stays in `out/duals` for every later step and layer, but each layer's clustering starts again from walks 1–5.
Walk size too coarse (uncommitted)	Paths averaged about 52 dual points (ids), so one walk had about 2,300 of them. `PATH_POINTS` = 10 now caps each path (`find_dual_points(max_points)`). That makes walks about 5 times smaller, so walks are added in finer steps and their dual points come from more starting points.
Interrupted runs lost hours 08caceb, 4147fb1, 7deaa46	• `layerL_clusters.p` holds the clusters, the rejected clusters, the seeds already tried and the walks in use, plus whether the layer is complete. • It is saved after every cluster, every 200 seeds and every new walk, and an interrupted run resumes from it. • Each step records its entry in `stats.json` before writing its output, so query totals stay correct across restarts. • A progress line is printed every 100 seeds.

7. Validation and diagnostics

- `validate.layer_error()` is printed after the solve, refine and signs stages of every layer. It reads the true weights for diagnostics only, and nothing from it feeds back into the attack. 7deaa46
- `match_layer` and `layer_error` leave out weights that are still exactly 0 (unknown), so rows aren't marked wrong for weights that haven't been recovered yet. eb7ae5a
- Helper scripts:
  - `stuck_neurons.py` samples a model to find neurons that are always on or always off.
  - `dual_neurons.py` attributes dual points to true neurons.
  - `linear_cluster.py` is an unused prototype of a different clustering approach, kept for reference.

  8b598bd, e17768f

8. Status and open items

- **test.keras:** on the last full check, every row and sign of layers 0–2 was recovered. Layer 3 has 15 of 16 neurons correct. The missing one, neuron 11, appears at no dual point in any walk, so the attack can't see it from the walks.
- **ids:** all 64 neurons of layers 0 and 1 were recovered. A layer-2 checkpoint held 62 valid clusters on the last check, and neuron 16 now clusters thanks to the triplet pruning fix. A full run to the end has not been confirmed yet.
- **Current ids run with capped walks** (in progress):
  - Layer 0 is done.
  - Layer 1 is clustering and has added walks 6–10, each about 400 dual points and 4.4M queries.
  - Smaller walks put fewer free experiments on each row. Layer 0's signs now cost 6.3M queries, against none with uncapped walks.

  Weigh this against the walk savings: raise `STORED_ENOUGH`, or make `PATH_POINTS` larger.
- **Seed order:** within a stage, seeds use up one walk before the next, while partners alternate between walks. Alternating the seeds too is a small, optional change.